current user, the property to be edited, and the `Edit` requirement. If `Succeeded` is `true`, the current user is authorized. If not, you return a `Forbid` result, which will redirect the user to the Access Denied endpoint. First, you need to inject the `IAuthorizationService` and capture it in a private field.

### Listing 10.27 Injecting `IAuthorizationService` into the `EditModel`

```
public class EditModel : PageModel
{
    private readonly IPropertyService _propertyService;
    private readonly ICityService _cityService;
    private readonly IAuthorizationService _authService;

    public EditModel(IPropertyService propertyService,
                     ICityService cityService,
                     IAuthorizationService authService)
    {
        _propertyService = propertyService;
        _cityService = cityService;
        _authService = authService;
    }
...
```

Then you modify the `OnGetAsync` method to perform the authorization check.

### Listing 10.28 Using `AuthorizeUserAsync` to evaluate user, resource, requirement

```
public async Task<IActionResult> OnGetAsync()
{
    var property = await _propertyService.FindAsync(Id);

    if (property == null)
    {
        return NotFound();
    }
    var result = await _authService.AuthorizeAsync(User,
    ➥ property, PropertyOperations.Edit);
    if (!result.Succeeded)
    {
```